Microsoft IQ Deep Dive with Python

July 28: Foundry IQ

July 29: Work IQ

July 30: Fabric IQ

Register at aka.ms/IQDeepDivePython/series

Microsoft IQ Deep Dive:
Fabric IQ

aka.ms/iqdeepdive/slides/fabriciq

Pamela Fox

Principal Cloud Advocate

Microsoft / GitHub

www.pamelafox.org

Today we'll cover...

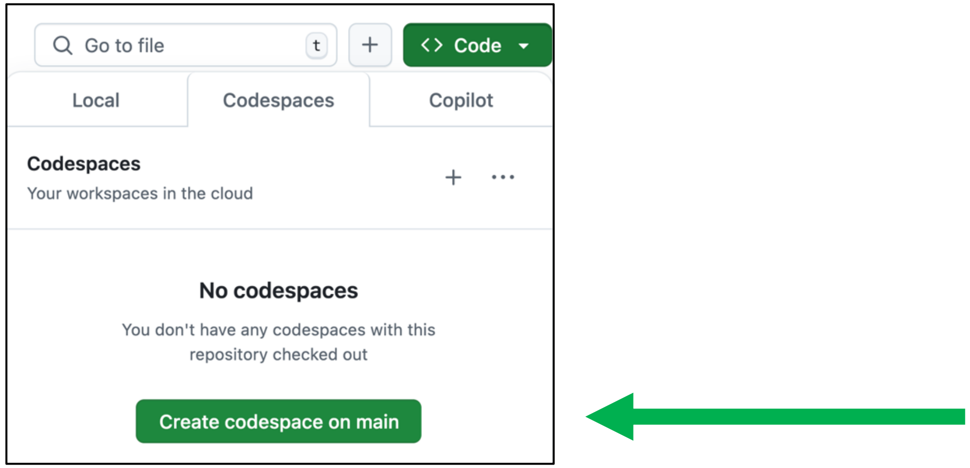
- Fabric IQ overview
- Ontologies
- Graphs
- Semantic models
- Data agents
- Fabric IQ + Foundry IQ

Want the code?

1. Open this GitHub repository:

https://aka.ms/iqdeepdive

2. Use "Code" button to create a GitHub Codespace:



3. Wait a few minutes for Codespace to start up ⏱️

⚠️ All code samples require deployment and an Azure account.

Microsoft IQ

Microsoft IQ Platform

Unified intelligence for enterprise AI

Work IQ	Fabric IQ	Web IQ	Foundry IQ
How your employees work	How your business operates	How you connect to web intelligence	How your agents unlock knowledge
Context on people, collaboration, and workflows	Context on business entities, systems of record, and actions	Context from the web, news, images and video	Context on policies, authoritative documents, and knowledge bases

Already covered: Session 2

Covering today!

Already covered: Session 1

Fabric IQ

Microsoft Fabric: One platform for all your data

Before Fabric:

Azure Data Lake

Azure SQL

Databricks

Power BI

Azure ML

5 services,
5 storage accounts,
5 auth models,
manual ETL between each pair.

With Microsoft Fabric:

- Data Factory
- Data Engineering
- Data Science
- Real-Time Intelligence
- Fabric IQ
- Power BI
- Databases
- Graph

OneLake

One shared storage for all workloads

Unify data in OneLake with zero ETL

Shortcut and mirroring sources

Generally available

New

- Azure SQL MI
- Azure Blob Storage
- Azure Cosmos DB
- Azure PostgreSQL
- SQL Server 2025
- SQL Server

- Azure SQL DB
- Azure Data Lake Store
- Microsoft OneLake
- Snowflake
- Google Cloud Storage
- Microsoft Dataverse
- Databricks Catalog
- Amazon S3
- S3 Compatible (cloud/on-prem)

Public preview

New

- SharePoint/ OneDrive
- SAP Datasphere

- Oracle DB
- Google BigQuery

Fabric IQ

Ontology

Shared business vocabulary of entities and relationships

Semantic Model

Curated business metrics, KPIs, and relationships for Power BI analytics

Graph

Analyze connections between entities and events

Data Agent

AI analyst that answers domain-specific questions

Plan

Collaborative planning, forecasting, and reporting

Digital Twin Builder

Create digital representations of real-world business entities and processes

Operations Agent

Monitors live data, detects issues, and remediates

OneLake

Unified data foundation for structured, unstructured, real-time, and graph data

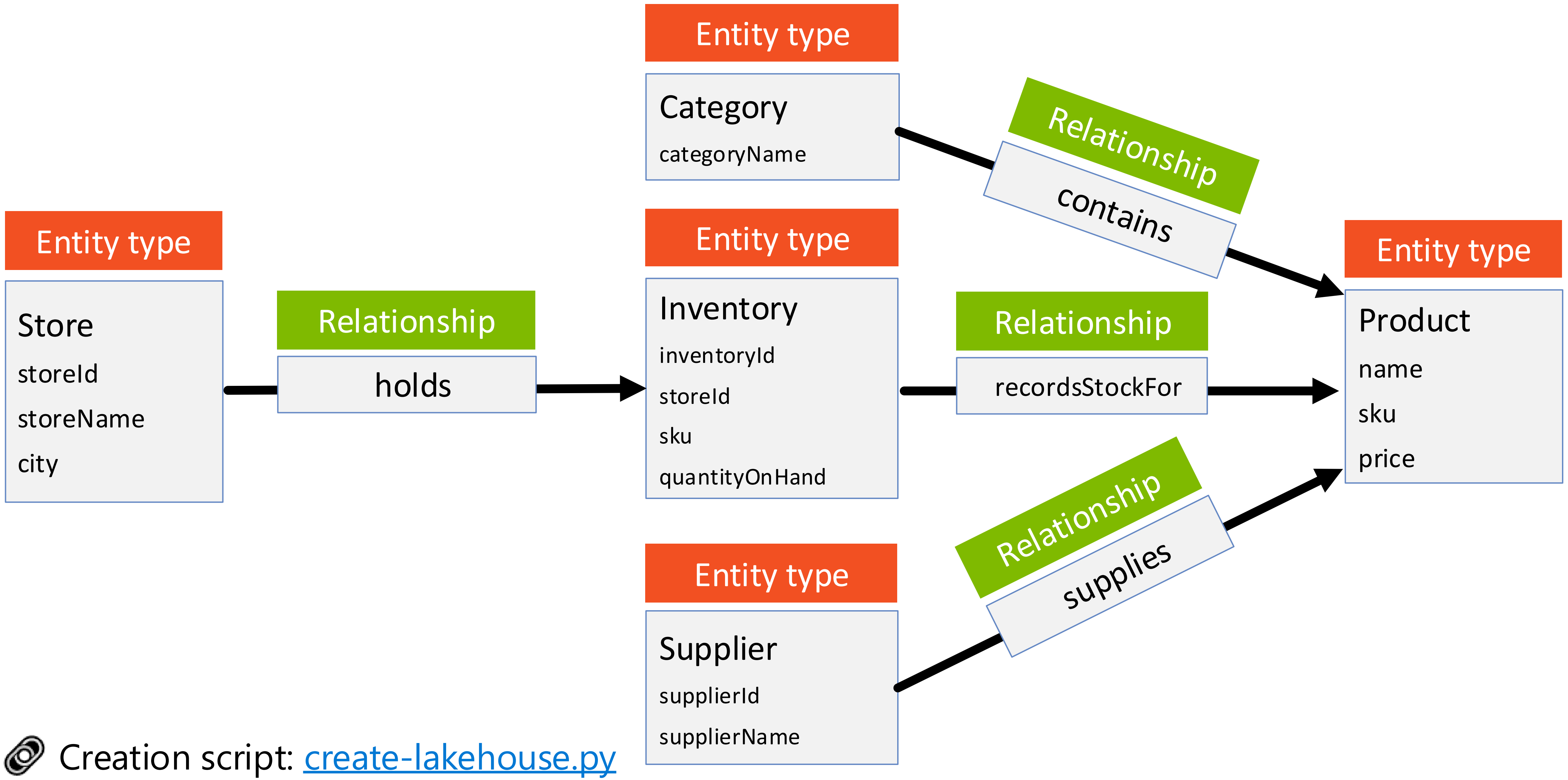
Fabric IQ: Ontology

Fabric IQ ontologies

An ontology is made up of:

Entity types	The concepts in your business (Product, Category, Store)
Properties	Named facts about an entity type (Product.name, Product.sku)
Relationships	Typed links between entity types (Category → contains → Product)
Data bindings	Connections to actual data in OneLake (Store.storeName → store.store_name)

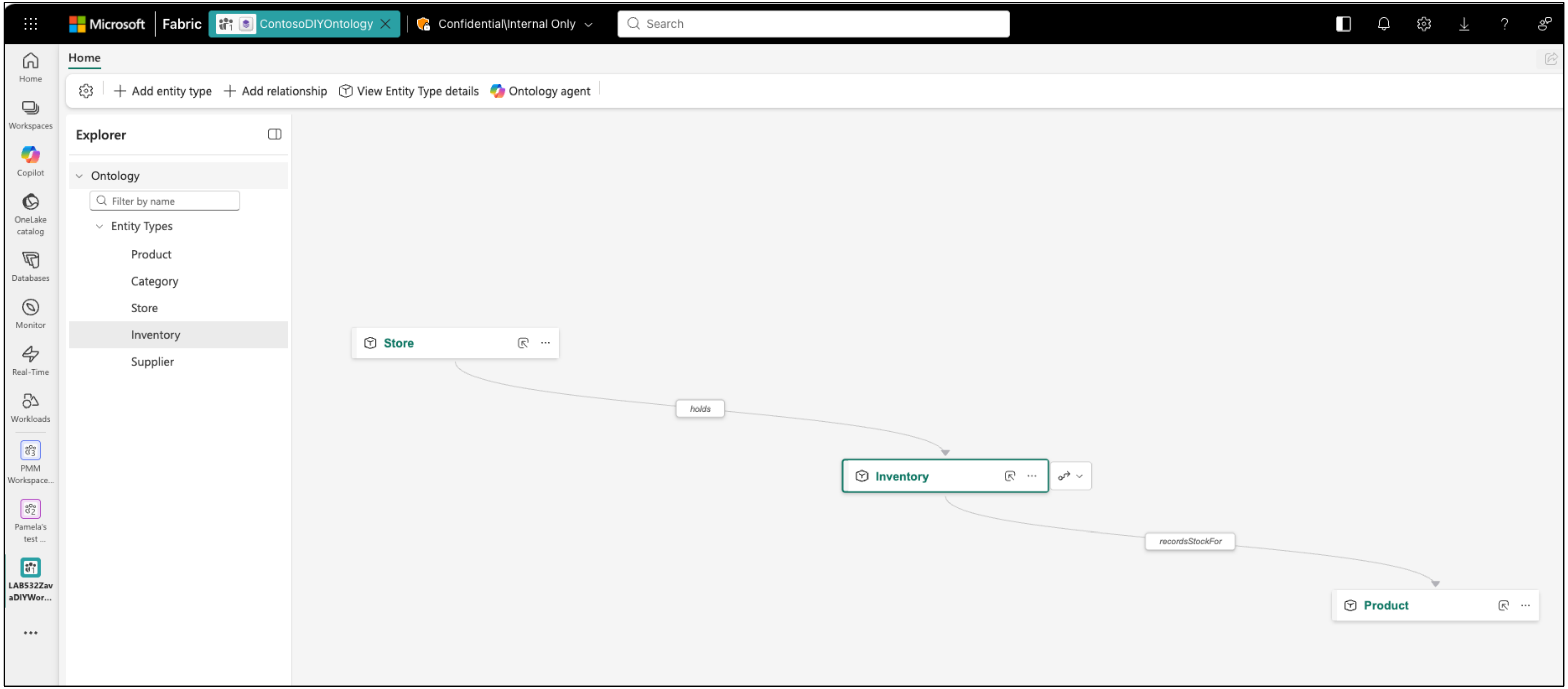
Ontology: Retail store



Creation script: [create-lakehouse.py]

Explore ontology in Fabric UI

DEMO



Query the ontology from agents

Many agents at Microsoft have built-in support for Fabric IQ ontologies:

- **Copilot Studio**
- **Fabric data agent**
- **Fabric operations agent**
- **Foundry IQ**
- **Foundry hosted agent + Foundry Toolbox**

Working with other agents?
Use the Ontology MCP server

https://learn.microsoft.com/fabric/iq/ontology/tutorial-4-create-data-agent

Ontology MCP server

```
https://api.fabric.microsoft.com/v1/mcp/dataPlane/workspaces/
WORKSPACE_ID/items/ONTOLOGY_ID/ontologyEndpoint
```

```
list_ontology_entity_types(
  includeProperties: Boolean)
```

→

```
{values: [{id, name, properties]}
```

```
search_ontology(
  naturalLanguageQuery: string,
  naturalLanguageResponse: boolean)
```

→

```
{raw: {},
 naturalLanguageResponse: "…"}
```

Full example: notebooks/fabriciq-ontology-mcp.ipynb

https://learn.microsoft.com/fabric/iq/ontology/how-to-use-ontology-mcp-server

Ontology MCP server + GitHub Copilot

DEMO

GitHub Copilot App

Query my Fabric ontology MCP server to see what types it has

I'll inspect the ontology schema and list its available entity types.

Fabric Ontology Mcp Server · List Ontology Entity Types

Arguments

```
{
  "includeProperties": true
}
```

{"values":[{"id":"1001","namespace":"usertypes","name":"Product","namespaceType":"Custom","entityIdParts":["100101"],"displayNamePropertyId":"100102","visibility":"Visible","properties":[{"id":"100101","name":"sku","valueType":"String"},{"id":"100102","name":"name","valueType":"String"},{"id":"100103","name":"category","valueType":"String"},{"id":"100104","name":"productType","valueType":"String"},{"id":"100105","name":"price","valueType":"Double"},{"id":"100106","name":"description","valueType":"String"},{"id":"100107","name":"stockLevel","valueType":"BigInt"},{"id":"100108","name":"imagePath","valueType":"String"},{"id":"1001

Your Fabric ontology has **5 custom entity types**:

Type	Key	Properties
Product	`sku`	9
Category	`categoryName`	13
Store	`storeId`	10
Inventory	`inventoryId`	9
Supplier	`supplierId`	5

All are visible, mapped to Lakehouse tables, and have no time-series properties or documents.

VS Code

what are the stock levels for hammers?

I'll query the Fabric ontology's `Product` entities for hammer products and return their current stock levels.

Ran `search_ontology` – fabric-iq-ontology (MCP Server)

Current stock levels for products in the **Hammers** category:

Product	SKU	Stock
Ball Peen Hammer 12oz	HTHM031200	204
Finishing Hammer 13oz	HTHM041300	492
Professional Claw Hammer 16oz	HTHM001600	368
Sledge Hammer 3lb	HTHM050300	308

Total hammer stock: 1,372 units.

The ontology also matched **Hammer Drill 1/2-inch**, with 371 units, but that product belongs to the **Drills** category rather than Hammers.

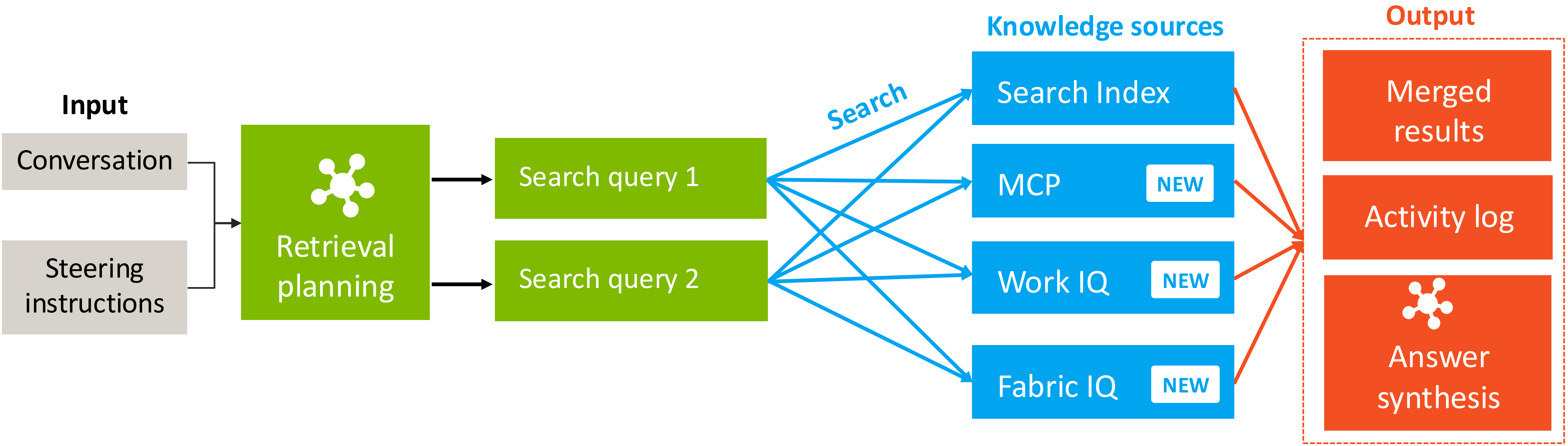
GPT-5.6 Sol • 5.2 credits

GitHub Copilot CLI

```
❯ what entities are in my fabric ontology?

● I'm checking the ontology tools first, then I'll list the entity types.

● list_ontology_entity_types (MCP: fabric-ontology-mcp-server) · include…

● Your fabric ontology has these entity types:

  • Product
  • Category
  • Store
  • Inventory
  • Supplier
```

Recap:

Foundry IQ knowledge base



Rewatch session 1 for more on Foundry IQ!

Foundry IQ with Fabric IQ knowledge source

Question

What are the stock levels for our hammers?

Ontology

Product:	name, sku, category
↑ recordsStockFor	
Inventory:	inventoryId, availableQuantity
↑ holds	
Store:	storeId, storeName

↓

OneLake Lakehouse

products	name, sku
inventory	inventory_id, available_quantity
stores	store_id, store_name

Results

sku:	HM13005
name:	12” Hammer
available_quantity:	50
category:	Hammer
store_id:	STORE-SEA

sku:	HM16325
name:	6” Hammer
available_quantity:	25
category:	Hammer
store_id:	STORE-SEA

https://learn.microsoft.com/azure/search/agentic-knowledge-source-how-to-fabric-ontology

Foundry IQ with Fabric ontology knowledge source

Create a Fabric Ontology knowledge source:

```
fabric_knowledge_source = FabricOntologyKnowledgeSource(
  name="fabric-ontology-knowledge-source", description="Operational data including stores, inventory, products",
  fabric_ontology_parameters=FabricOntologyKnowledgeSourceParameters(
    workspace_id=FABRIC_WORKSPACE_ID, ontology_id=FABRIC_ONTOLOGY_ID))
```

Create a knowledge base that includes the Fabric Ontology knowledge source:

```
knowledge_base = KnowledgeBase(
  name="multisource-fabric-ontology-knowledge-base",
  description="Multi-source knowledge base combining indexed company documents and product data",
  models=[KnowledgeBaseAzureOpenAIModel(azure_open_ai_parameters=aoai_params)],
  knowledge_sources=[KnowledgeSourceReference(name="fabric-ontology-knowledge-source"),
                     KnowledgeSourceReference(name="hrdocs-knowledge-source")],
  retrieval_instructions="Use Fabric Ontology for products. Use search indexes for HR and health policy docs.")
```

Query the knowledge base with an authenticated user token and user query:

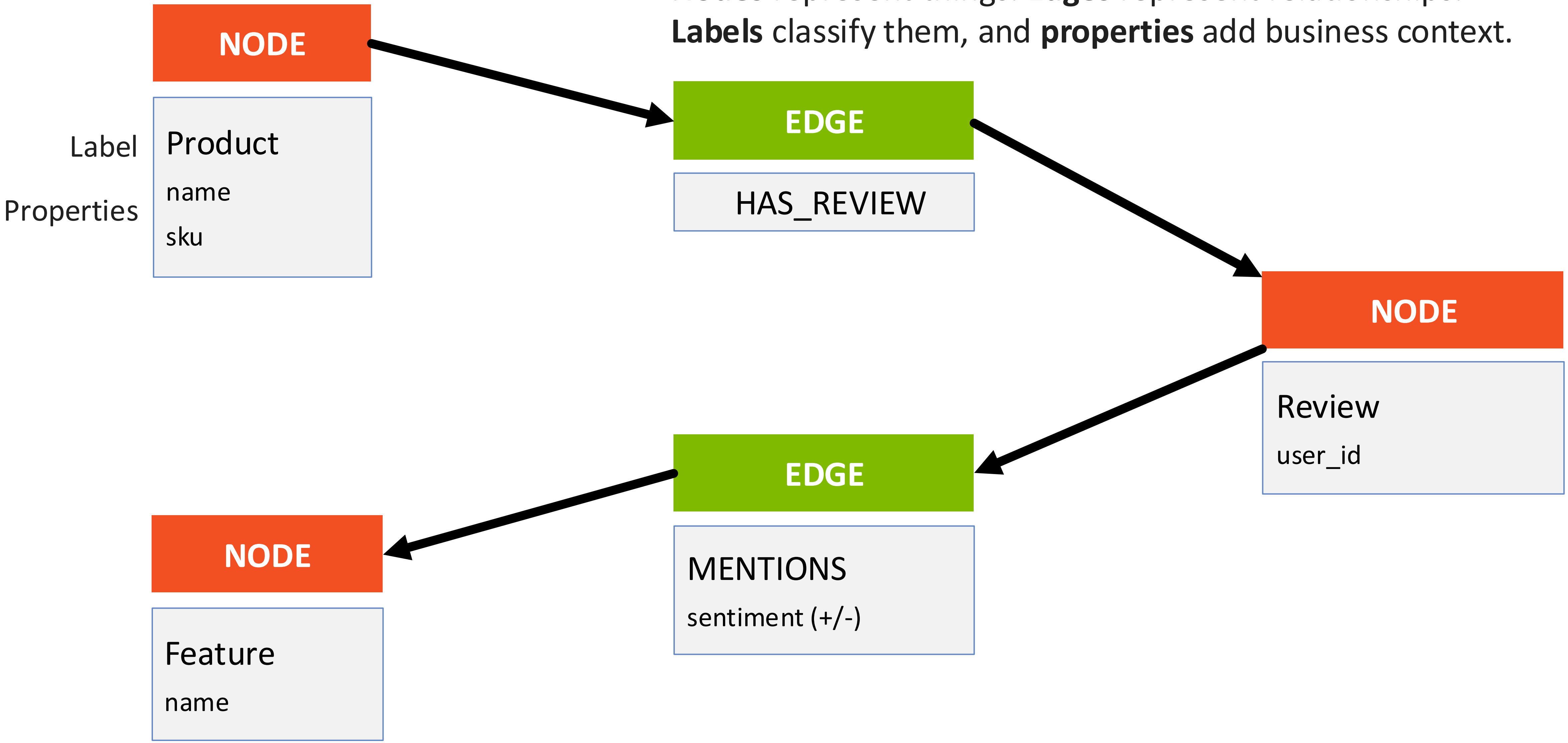
```
result = knowledge_base_client.retrieve(KnowledgeBaseRetrievalRequest(
     messages=[KnowledgeBaseMessage(role="user", content=[KnowledgeBaseMessageTextContent(text=question)])],
    query_source_authorization=user_token)
```

🔗 Full example: notebooks/foundryiq-fabriciq-ontology.ipynb

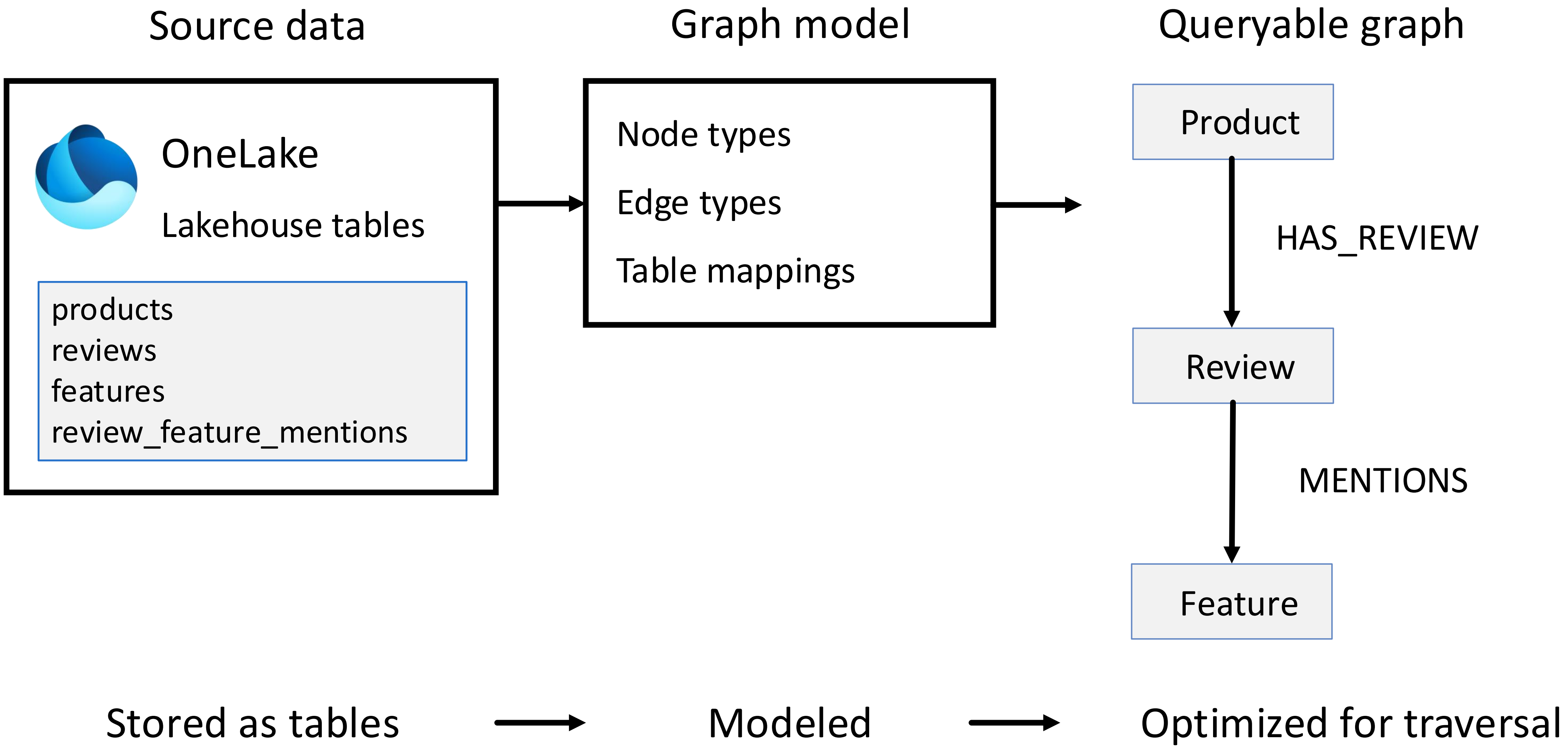
Fabric IQ: Graph

Fabric IQ graphs

Nodes represent things. **Edges** represent relationships. **Labels** classify them, and **properties** add business context.

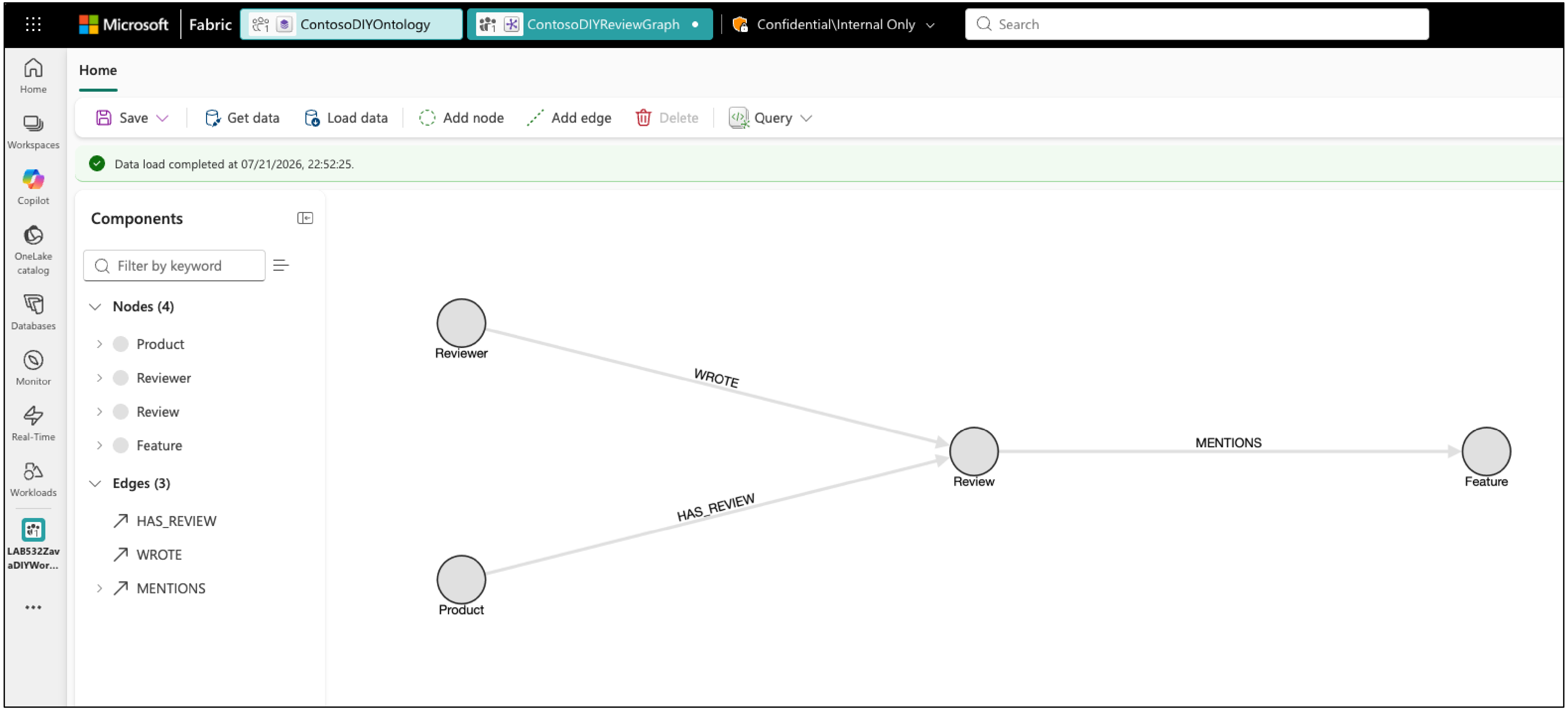


From OneLake to a queryable graph



Explore graph in Fabric UI

DEMO



Query the graph with GQL

GQL query

```
MATCH (p:`Product`)
  -[:`HAS_REVIEW`]->
  (r:`Review`)
  -[m:`MENTIONS`]->
  (f:`Feature`)
FILTER m.sentiment = 'negative'
RETURN p.name AS productName,
       r.rating AS rating,
       f.featureName AS feature,
       m.sentiment AS sentiment,
       m.confidence AS confidence
ORDER BY confidence DESC
LIMIT 20
```

Full example: notebooks/fabriciq-graph.ipynb

GQL over REST

```
POST …/executeQuery?preview=true

{
   "query": "MATCH ..."
}

Authorization: Bearer token
```

https://learn.microsoft.com/fabric/graph/gql-query-api

Query the graph from agents

For NL2GQL (Natural Language to Graph Query Language), add the graph as a data source to **Fabric data agent.**

Which products should be recommended together because they are frequently bought by similar customers who make purchases in the same countries and buy from the same categories or subcategories?

NL2GQL

Gql

```
-- Customer → first order → product context
MATCH (node_customer:`customer`)-[edge_placed1:`placed`]->(node_order1:`order`)-
[edge_contains1:`contains`]->(node_product1:`product`)-[edge_belongs_to1:`belongs_to`]->
(node_subcategory1:`subcategory`)-[edge_under1:`under`]->(node_category1:`category`)
-- Order → employee → country
MATCH (node_order1)<-[:processed]-(node_employee:`employee`)-[:lives_in]->
(node_country:`country`)
-- Same customer → second order → second product
MATCH (node_customer)-[edge_placed2:`placed`]->(node_order2:`order`)-[edge_contains2:`contains`]-
>(node_product2:`product`)-[edge_belongs_to2:`belongs_to`]->(node_subcategory2:`subcategory`)-
[edge_under2:`under`]->(node_category2:`category`)
WHERE node_order1.`salesOrderDetailID` <> node_order2.`salesOrderDetailID`
  AND node_country.`country` IS NOT NULL
  AND (node_category1.`categoryID` = node_category2.`categoryID` OR node_subcategory1.`subCategor
yID` = node_subcategory2.`subCategoryID`)
  AND node_product1.`productID` <> node_product2.`productID`
LET prod1 = node_product1.`productName`, prod2 = node_product2.`productName`, cat = node_category
1.`categoryName`, ctry = node_country.`country`
RETURN prod1, prod2, cat, ctry, COUNT(*) AS pairCount
GROUP BY prod1, prod2, cat, ctry
ORDER BY pairCount DESC
LIMIT 1000
```

https://blog.fabric.microsoft.com/blog/graph-powered-ai-reasoning-preview/

Fabric IQ: Semantic models

Fabric IQ semantic models

A tabular analytical model optimized for aggregations and Power BI reporting

Tables — The business entities in your model
(e.g. Sessions, Page Views, Channels, Date)

Relationships — How tables connect to each other
(e.g. Page Views → Sessions, Sessions → Date)

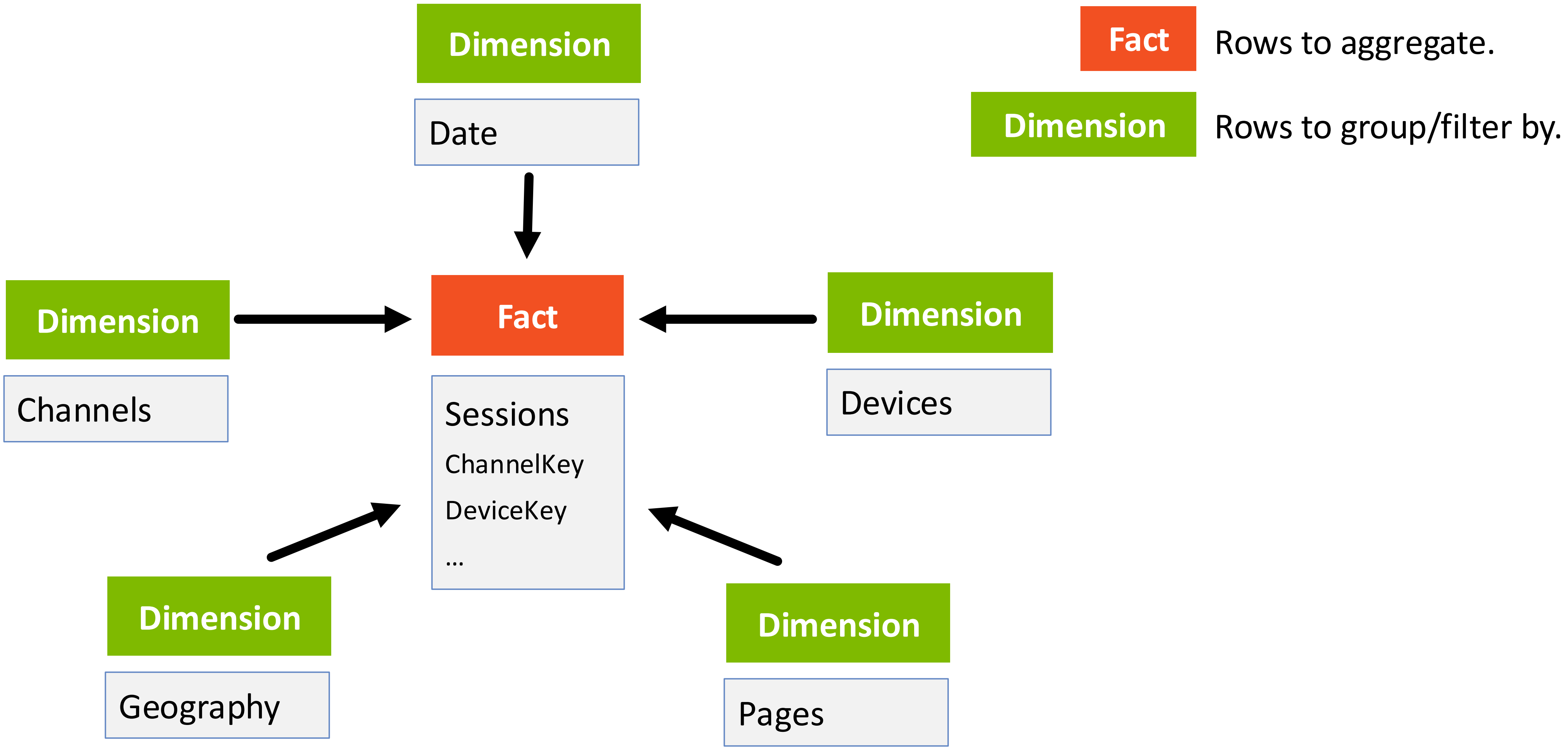
Measures — Reusable DAX calculations
(e.g. Total sessions, Bounce rate)

Data source — DirectLake connection to OneLake data
(e.g. Lakehouse SQL endpoint)

→ **Power BI reports**

Queries the models directly

Example: Web analytics star schema



Defining semantic models with TMDL

TMDL is a declarative format for Power BI semantic models in Fabric

Fact table

```
table Sessions
 partition Sessions = entity
  mode: directLake
  source
    entityName: fact_sessions
    schemaName: dbo
    expressionSource: DatabaseQuery

 measure 'Total Sessions' = COUNTROWS(Sessions)
 measure 'Revenue' = SUM(Sessions[RevenueAmount])
```

Models: data/semantic-models/web-analytics

Dimension table

```
table Date
 column DateKey dataType: int64 sourceColumn: date_key
 column Date dataType: dateTime sourceColumn: date
 column Year dataType: int64 sourceColumn: year
 column Month dataType: string sourceColumn: month_name

 partition Date = entity
  mode: directLake
  source
   entityName: dim_dates
   schemaName: dbo
   expressionSource: DatabaseQuery
```

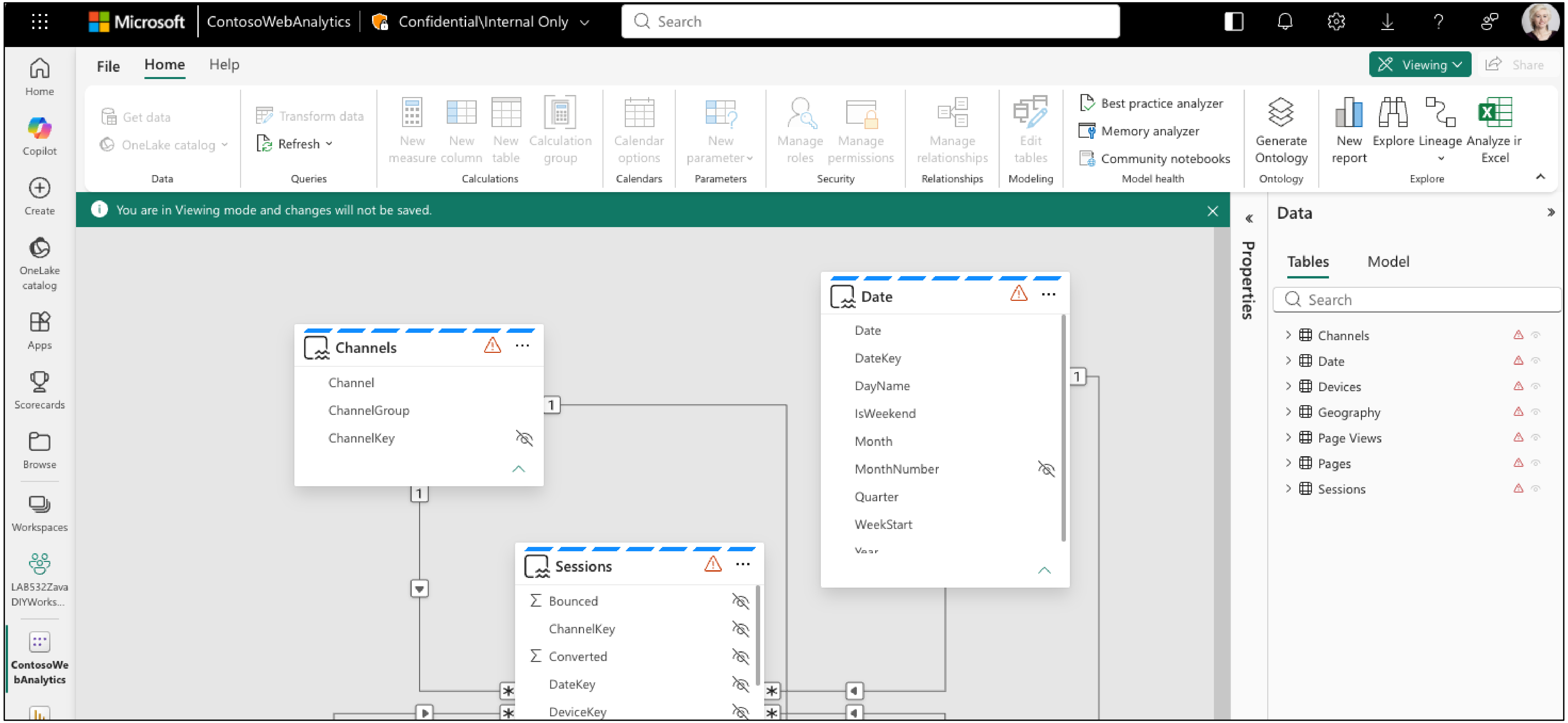
Create models programmatically with Fabric SDK:

```
semantic_model = client.semanticmodel.items.create_semantic_model(FABRIC_WORKSPACE_ID,
  CreateSemanticModelRequest(display_name=SEMANTIC_MODEL_NAME, definition=definition))
```

Full code: infra/create-semantic-model.py

Explore semantic models in Fabric UI

DEMO



Anatomy of a Power BI report

Reports contain pages; pages contain visuals; visuals bind to semantic models.

Report Web Analytics Dashboard

Page webAnalyticsOverview

Visual Card: Bounce Rate

Visual Card: Revenue

Visual Line chart: Sessions over time

Filter Date range = Last 30 days

PBIR JSON definitions: data/reports/web-analytics

Creating Power BI reports with Fabric SDK

Define report components in PBIR:

```
{
  "$schema":
"https://developer.microsoft.com/json-
schemas/fabric/item/report/definitionProp
erties/2.0.0/schema.json",
  "version": "4.0",
  "datasetReference": {
    "byConnection": {
      "connectionString":
"semanticmodelid={{SEMANTIC_MODEL_ID}}"
    }
  }
}
```

PBIR: data/reports/web-analytics

Bind PBIR to semantic models and publish report:

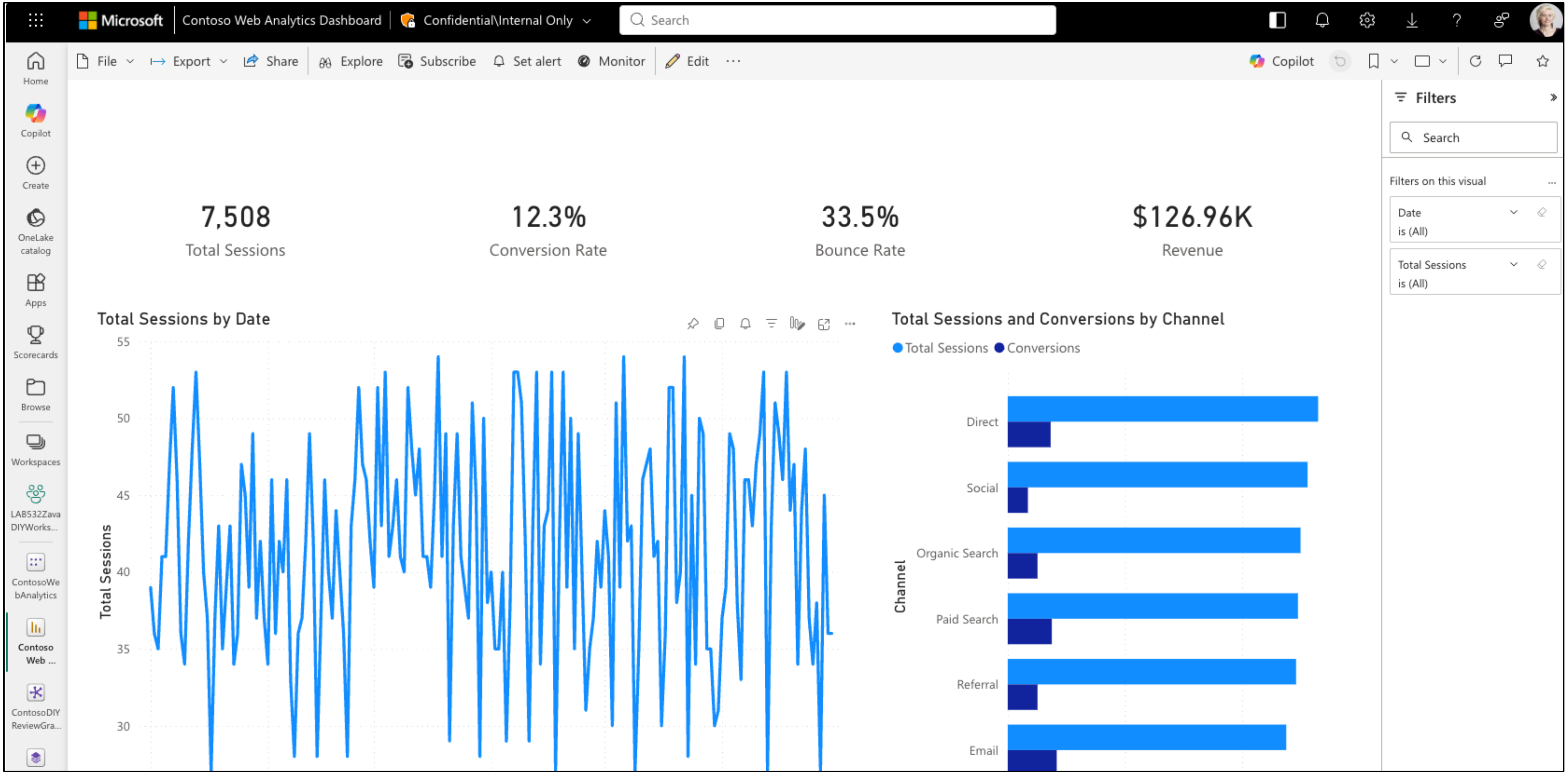
```
parts = [ReportDefinitionPart(
  path="definition.pbir",
  payload=base64.b64encode(
    pbir_text.replace("{{SEMANTIC_MODEL_ID}}",
      semantic_model_id).encode()).decode(),
  payload_type="InlineBase64")
]

report = client.report.items.create_report(
 workspace_id,
 CreateReportRequest(
  display_name=REPORT_NAME,
  definition=ReportDefinition(format="PBIR", parts=parts)))
```

Full code: infra/create-web-analytics-report.py

The PBIR manifest binds the report to the semantic model. Pages, visuals, and filters are separate definition parts. The Fabric SDK uploads all parts as a single report definition.

Power BI reports in Fabric UI

DEMO



Query the semantic models from agents

For NL2DAX (Natural Language to DAX),
add the semantic models as a data source to **Fabric data agent.**

What were the top five pages by session count last week, and which device category drove most of those sessions?

NL2DAX →

```
EVALUATE
SUMMARIZECOLUMNS(
    'Pages'[PageTitle],
    'Devices'[DeviceCategory],
    'Date'[Week],
    "Sessions", [Total Sessions]
)
ORDER BY [Sessions] DESC
```

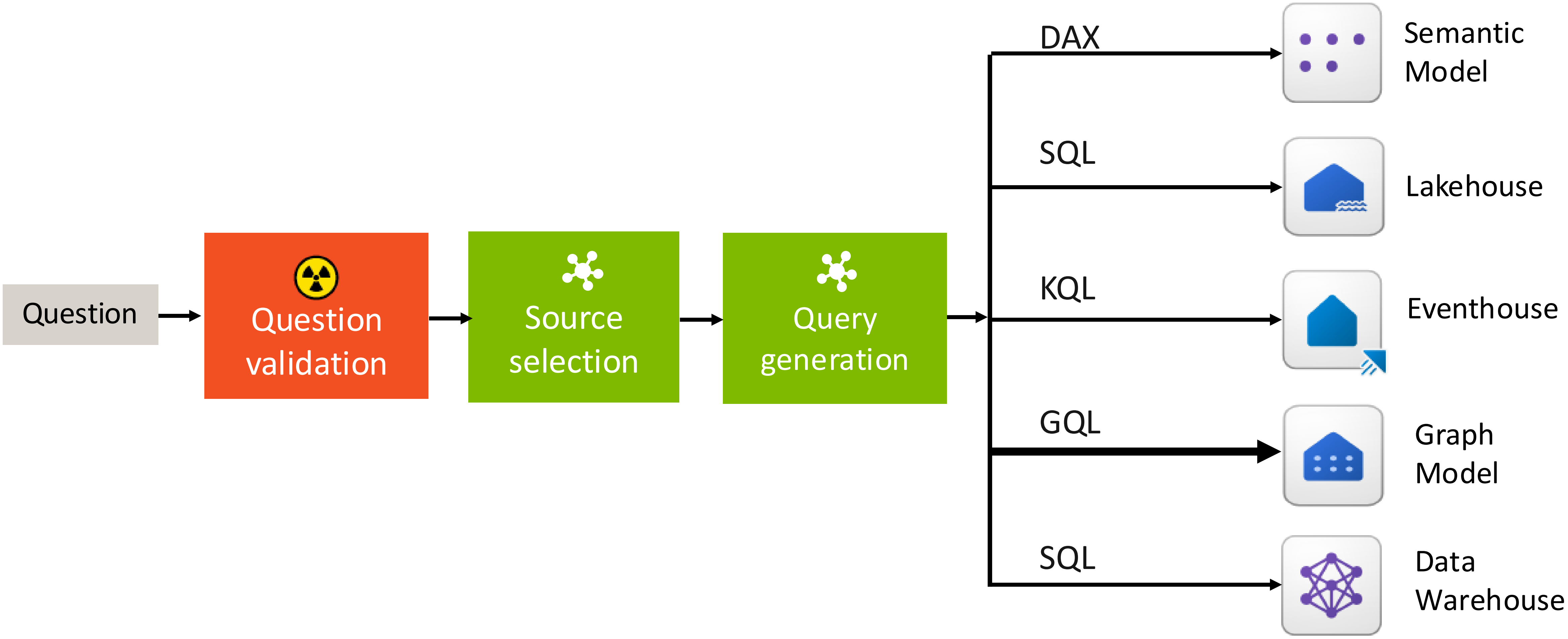
To make NL2DAX accurate: model as a star schema, use business-friendly names, define explicit DAX measures, and configure an AI data schema using Prep for AI.

https://learn.microsoft.com/fabric/data-science/semantic-model-best-practices

Fabric IQ: Data agent

Fabric data agent

A conversational agent that can query across multiple Fabric data sources:



Data agent MCP server

```
https://api.fabric.microsoft.com/v1/mcp/workspaces/
WORKSPACE_ID/dataagents/DATA_AGENT_ID/agent
```

```
DataAgent_YourDataAgentName(
  userQuestion: string)
```

→

```
"The answer to your question…"
```

🔗 Full example: notebooks/fabriciq-dataagent-mcp.ipynb

https://learn.microsoft.com/fabric/data-science/data-agent-mcp-server

Data Agent MCP server + GitHub Copilot

DEMO

Current | Sessions | Issues | Pull requests | Gists

Copilot v1.0.72-1 uses AI.
/experimental

- Tip: /mcp
  └ Manage MCP server configuration

- MCP Servers reloaded: 10 servers connected

❯ What are the total website revenue and conversion rate? 22:58

- I'll pull those metrics from the Contoso DIY data agent now.

- **DataAgent_ContosoDIYDataAgent** (MCP: fabric-data-agent-mcp-server)
  userQuestion: "What are the total website revenue and conversion rate?"

  The total website revenue is $126,956.69, and the overall conversion rate is 12.3%.

{"deepLinkUrl":"]8;id=md-155yqt8;https://msit.fabric.microsoft.com/groups/f200c1bc-e-9308-be868f68753e/aiskills/eeb78c02-6ae8-4d94-8d93-4476e468650e/stage/published/threads/thread_i6d7xmeRO5Wofe6F1iXIqi0R/runs/run_fabRN6d6H41avrtb98ZdfelwxPRa/question/msg_GYPbI9PgQMCFGXeNgnPUF5xY/source/mcs","artifactName":"ContosoDIYDataAgent"}https://sit.fabric.microsoft.com/groups/f200c1bc-e2ec-43b9-9308-be868f68753e/aiskills/eeb78c02-6ae8-4d94-8d93-4476e468650e/stage/published/threads/thread_i6d7xmeRO5Wofe6F1iXIqi0R/runs/run_fabRN6d6H41avrtb98ZdfelwxPRa/question/msg_GYPbI9PgQMCFGXeNgnPUF5xY/source/mcs","artifactName":"ContosoDIYDataAgent"}

- The **total website revenue is $126,956.69** and the **overall conversion rate is 12.3%.**

GitHub Copilot CLI

What are the total website revenue and conversion rate?

I'm pulling the website-wide totals from the analytics data source.

Fabric Data Agent Mcp Server · DataAgent ContosoDIYDataAgent

Arguments

```
{
  "userQuestion": "What are the total website revenue and overall website conversion rate? Use the full available date range unless a default reporting period is defined. State the revenue total, conversion rate percentage, and the calculation basis/date range."
}
```

- Total Website Revenue: $126,956.69
- Overall Website Conversion Rate: 12.3%
- Calculation Basis/Date Range: From 2026-MM-01 to 2026-MM-30 (the full available reporting period)

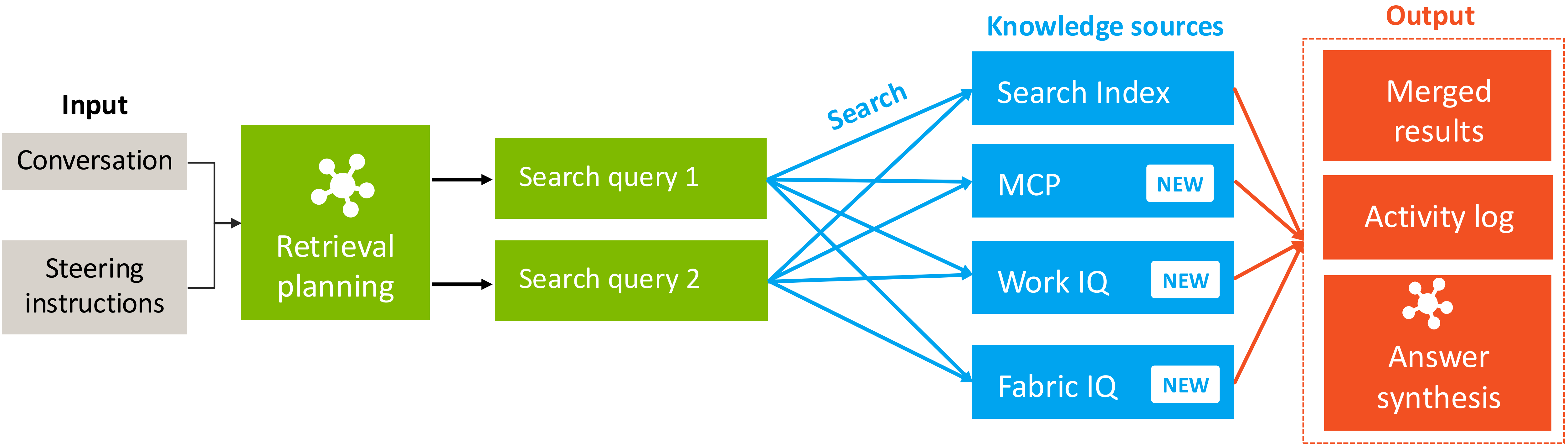
If you need more specific breakdowns or clarification regarding the exact months involved, let me know!

Total website revenue: $126,956.69
Conversion rate: 12.3%

GitHub Copilot App

Recap:

Foundry IQ knowledge base



Rewatch session 1 for more on Foundry IQ!

Foundry IQ with Fabric data agent knowledge source

Question

What are the stock levels for our hammers?

→

Fabric data agent

Ontology	Product inventories
Graph	Product reviews
Semantic models	Website analytics

↓

Ontology

Product:	name, sku, category
Inventory:	inventoryId, availableQuantity
Store:	storeId, storeName

↓

OneLake Lakehouse

products	name, sku
inventory	inventory_id, available_quantity
stores	store_id, store_name

→

Results

sku:	HM13005
name:	12” Hammer
available_quantity:	50
category:	Hammer
store_id:	STORE-SEA

sku:	HM16325
name:	6” Hammer
available_quantity:	25
category:	Hammer
store_id:	STORE-SEA

Foundry IQ with Fabric data agent knowledge source

Create a Fabric Data Agent knowledge source:

```
fabric_knowledge_source = FabricDataAgentKnowledgeSource(
  name="fabric-data-agent-knowledge-source", description="Product inventory, reviews, website analytics",
  fabric_data_agent_parameters=FabricDataAgentKnowledgeSourceParameters(
    workspace_id=FABRIC_WORKSPACE_ID, data_agent_id=FABRIC_DATA_AGENT_ID))
```

Create a knowledge base that includes the Fabric Data Agent knowledge source:

```
knowledge_base = KnowledgeBase(
  name="multisource-fabric-data-agent-knowledge-base",
  description="Multi-source knowledge base combining indexed company documents with Fabric data agent.",
  models=[KnowledgeBaseAzureOpenAIModel(azure_open_ai_parameters=aoai_params)],
  knowledge_sources=[KnowledgeSourceReference(name="hrdocs-knowledge-source"),
                     KnowledgeSourceReference(name="fabric-data-agent-knowledge-source")],
  retrieval_instructions="Use data agent for product, inventory, analytics. Use search index for HR docs.")
```

Query the knowledge base with an authenticated user token and user query:

```
result = knowledge_base_client.retrieve(
   KnowledgeBaseRetrievalRequest(
     messages=[KnowledgeBaseMessage(role="user", content=[KnowledgeBaseMessageTextContent(text=question)])],
   query_source_authorization=user_token)
```

Full example: notebooks/foundryiq-fabriciq-dataagent.ipynb

Microsoft IQ Platform

Unified intelligence for enterprise AI

Work IQ

How your
employees work

Context on people, collaboration, and workflows

Fabric IQ

How your
business operates

Context on business entities, systems of record, and actions

Web IQ

How you connect
to web intelligence

Context from the web, news, images and video

Foundry IQ

How your agents
unlock knowledge

Context on policies, authoritative documents, and knowledge bases

Microsoft

Reactor

JOIN US LIVE

Microsoft IQ Live

Bi-weekly episodes on Microsoft Reactor, from Aug 6 to Nov 12, 2026.

Register → aka.ms/MicrosoftIQLive

Follow Microsoft Reactor on YouTube to catch every stream.

Next steps

Join office hours after in Discord:
aka.ms/pythonai/oh

July 28: Foundry IQ

July 29: Work IQ

July 30: Fabric IQ

Register at aka.ms/IQDeepDivePython/series